Lab 04

Lakhvinder Singh

04.08.2026

Lab 04: Raster data management and analysis

Read the instructions COMPLETELY before starting the lab

This lab builds directly on the raster analysis concepts we worked through in class, including the Week 12 in-class notebook and the Wildcard Thursday activity. You should have that code available for reference.

Formatting your submission

This lab must be placed into a public repository on GitHub (www.github.com). Before the due date, submit **on Canvas** a link to the repository. I will then download your repositories and run your code. The code must be contained in either a .R script or a .Rmd markdown document. As I need to run your code, any data you use in the lab must be referenced using **relative path names**. Finally, answers to questions I pose in this document must also be in the repository at the time you submit your link to Canvas. They can be in a separate text file, or if you decide to use an RMarkdown document, you can answer them directly in the doc.

Data

All data for this lab are in the course repository at:

`../data/wildcard_thursday/erie_raster_ex/`

You will find two subdirectories:

- `images/` — seven weekly composite GeoTIFF rasters of cyanobacteria bloom intensity in Lake Erie (summer–fall 2017)
- `bounds/` — several shapefiles representing geographic areas of interest within the Lake Erie region

Before writing any code, read the metadata file at

`../data/wildcard_thursday/erie_raster_ex/erie_README.txt.`

This is not optional. Understanding your data source, its scaling, and its data key is essential for correct analysis.

Introduction

Cyanobacteria (blue-green algae) blooms in Lake Erie have become a significant environmental and public health concern. These blooms can produce toxins that contaminate drinking water and harm aquatic ecosystems. The data you will use were produced by NOAA using imagery from the European Space Agency's Sentinel-3 satellite. Each raster represents the maximum observed bloom intensity during a given week.

The product you will be working with is the **CIcyano** (Cyanobacteria Index, cyano only) product. Values in these rasters are stored as **digital numbers (DN)** and must be transformed to meaningful physical units before interpretation. You saw this in class — recall the reverse-scaling formula from the README and from the Week 12 notebook.

A critical aspect of working with remote sensing data is understanding **sentinel values**: special numeric codes that indicate conditions like cloud cover, land pixels, or missing data. These are NOT real observations and must be handled carefully before any analysis.

We begin by loading the relevant packages:

```
library(terra)
```

```
## terra 1.9.1
```

```
library(sf)
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.1     v tibble    3.3.1
## v lubridate 1.9.4     v tidyr     1.3.2
## v purrr     1.2.1
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x tidyr::extract() masks terra::extract()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tmap)
library(fs)
```

Task 1: Data management and preprocessing

1.1 Inventory your rasters

Use `fs::dir_ls` with the parameter `regexp = "\\.tif$"` to get the file paths for all raster images in the `images/` directory. Store the result in an object called `raster_files`. Print the result — how many files are there?

```
raster_files <- fs::dir_ls(
  path = "./erie_raster_ex/images/",
  regexp = "\\.tif$"
)

#Print the result
print(raster_files)
```

```
## ./erie_raster_ex/images/ts_2017.0802_0808.L4.LE3.CIcyano.MAXIMUM_7day.tif
## ./erie_raster_ex/images/ts_2017.0816_0822.L4.LE3.CIcyano.MAXIMUM_7day.tif
## ./erie_raster_ex/images/ts_2017.0823_0829.L4.LE3.CIcyano.MAXIMUM_7day.tif
## ./erie_raster_ex/images/ts_2017.0920_0926.L4.LE3.CIcyano.MAXIMUM_7day.tif
## ./erie_raster_ex/images/ts_2017.0927_1003.L4.LE3.CIcyano.MAXIMUM_7day.tif
## ./erie_raster_ex/images/ts_2017.1018_1024.L4.LE3.CIcyano.MAXIMUM_7day.tif
## ./erie_raster_ex/images/ts_2017.1025_1031.L4.LE3.CIcyano.MAXIMUM_7day.tif
```

```
#Check how many files?
length(raster_files)
```

```
## [1] 7
```

```
#There are 7 files
```

1.2 Inspect a single raster

Load the first raster in your list using `terra::rast()`. Examine the following properties and record what you find:

- Coordinate reference system: `terra::crs()`
- Spatial extent: `terra::ext()`
- Spatial resolution: `terra::res()`
- Number of layers: `terra::nlyr()`
- Range of cell values (use `terra::global()` with `"min"` and `"max"`)

```
#first raster from list
first_raster <- terra::rast(raster_files[1])

#coordinate reference system
terra::crs(first_raster)
```

```
## [1] "PROJCRS[\"WGS 84 / UTM zone 17N\",\n    BASEGEOGCRS[\"WGS 84\",\n        DATUM[\"World Geodetic
```

```
#spatial extent
terra::ext(first_raster)
```

```
## SpatExtent : 281282.934996, 694382.934996, 4572608.997997, 4764308.997997 (xmin, xmax, ymin, ymax)
```

```
#spatial resolution
terra::res(first_raster)
```

```
## [1] 300 300
```

```
#number of layers
terra::nlyr(first_raster)
```

```
## [1] 1
```

```
#range of cell values
terra::global(first_raster, fun = c("min", "max"), na.rm = TRUE)
```

```
##                                                min max
## ts_2017.0802_0808.L4.LE3.CIcyano.MAXIMUM_7day   1 252
```

1.3 Write a preprocessing function: `clean_cicyano()`

Looking at the data key in the README, you will notice that values of 0 and 250 and above have special meanings — they are **not** valid bloom observations. Before doing any analysis, these sentinel values must be replaced with `NA`.

Write a function called `clean_cicyano(r)` that:

1. Takes a SpatRaster `r` as input
2. Builds a reclassification matrix that maps **0** and **all values >= 250** to `NA`, and leaves values 1–249 unchanged
3. Applies the reclassification using `terra::classify()` with the argument `others = NA` (this is a useful argument — look it up)
4. Returns the cleaned raster

Hint: Your reclassification matrix should handle the no-detection case (0) and all sentinel codes (250 through 255). Review how `terra::classify()` uses a three-column matrix to specify `[from, to, becomes]` intervals.

```r
#preprocessing function
clean_cicyano <- function(r) {

  m <- matrix(c(
    -0.5,   0.5, NA,  # Captures 0 and turns it to NA
    249.5, 255.5, NA  # Captures 250 through 255 and turns them to NA
  ), ncol = 3, byrow = TRUE)

  #reclassification
  # (By targeting only the NAs, values 1-249 are left unchanged)
  clean_r <- terra::classify(r, rcl = m)

  #cleaned raster
  return(clean_r)
}

# Running the first raster through the cleaning function
first_raster_clean <- clean_cicyano(first_raster)

# Comparing the global min/max of the raw vs. cleaned raster
print("Raw Raster Min/Max:")
```

```
## [1] "Raw Raster Min/Max:"
```

```r
terra::global(first_raster, fun = c("min", "max"), na.rm = TRUE)
```

```
##                                               min max
## ts_2017.0802_0808.L4.LE3.CIcyano.MAXIMUM_7day   1 252
```

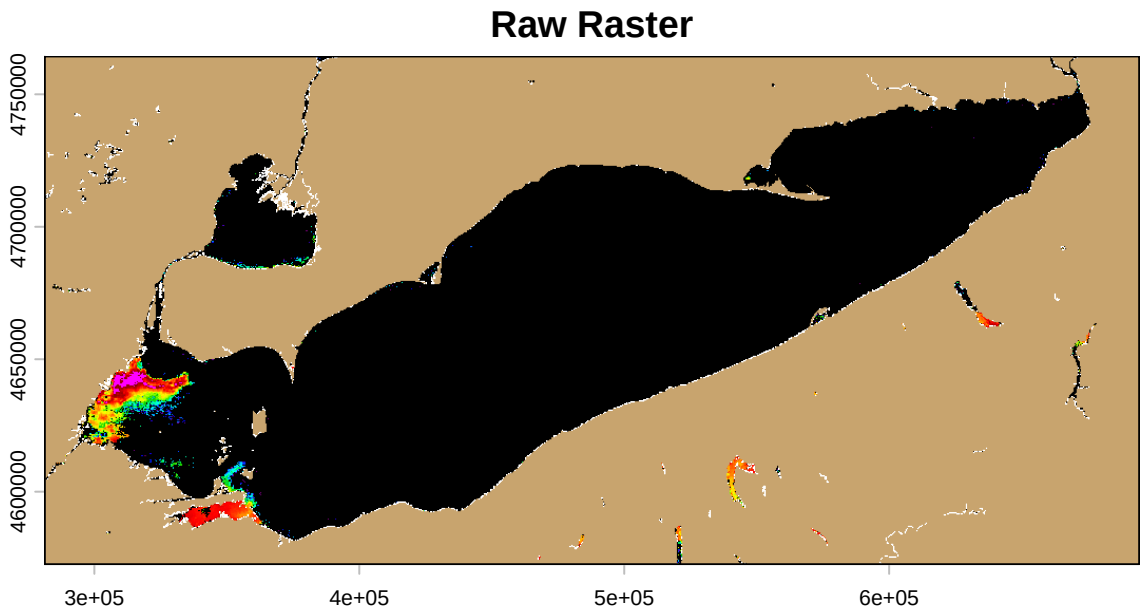
```r
print("Cleaned Raster Min/Max:")
```

```
## [1] "Cleaned Raster Min/Max:"
```

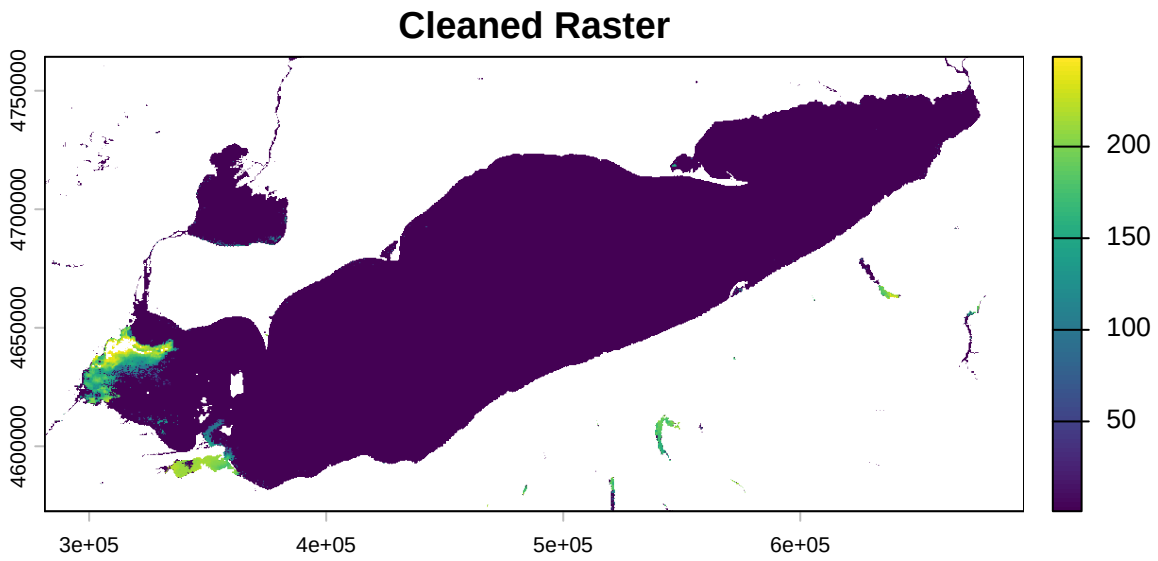
```r
terra::global(first_raster_clean, fun = c("min", "max"), na.rm = TRUE)
```

```
##                                               min max
## ts_2017.0802_0808.L4.LE3.CIcyano.MAXIMUM_7day   1 249
```

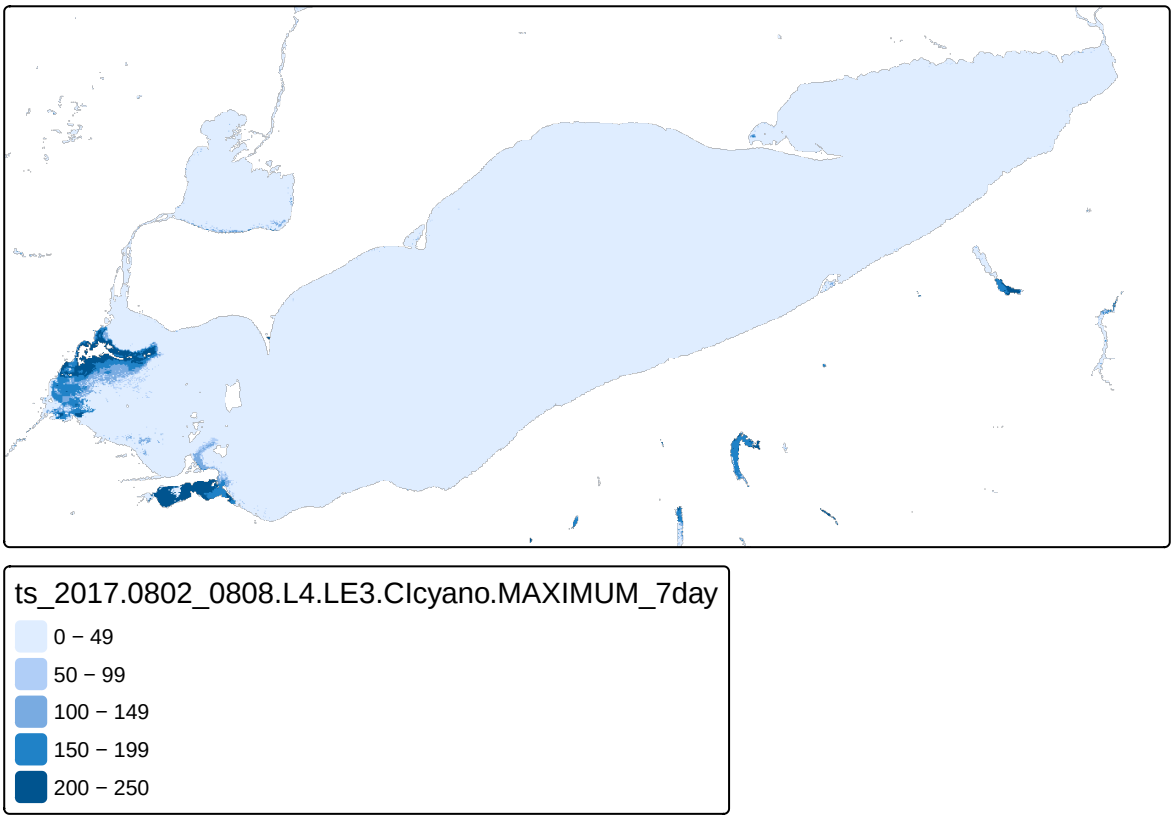
```r
#raw raster plot
plot(first_raster, main = "Raw Raster")
```




```
#cleaned raster plot
plot(first_raster_clean, main = "Cleaned Raster")
```




```
#plot using tmap
qtm(first_raster_clean)
```



Test your function on the first raster. Verify it worked by comparing the global min/max of the raw raster vs. the cleaned raster.

1.4 Write a transform function: `dn_to_ci()`

The DN values in these rasters are scaled according to the formula in the README. To convert back to physical CI units, use the reverse-scaling formula:

$$CI = 10^{\left(\frac{3.0}{250.0} \times DN - 4.2\right)}$$

Write a function called `dn_to_ci(r)` that takes a cleaned SpatRaster and returns a new raster with values transformed to CI units. You can apply a function to a raster using standard arithmetic operations or by passing a function to the raster.

```
#transformation function
dn_to_ci <- function(r) {

  # reverse-scaling formula applied directly to the SpatRaster
  ci_raster <- 10^(((3.0 / 250.0) * r) - 4.2)

  return(ci_raster)
}

#Applying the function to the cleaned raster from Task 1.3
first_raster_ci <- dn_to_ci(first_raster_clean)

#Checking the range of the physical CI units
print("CI Raster Min/Max:")
```

```
## [1] "CI Raster Min/Max:"
```

```
terra::global(first_raster_ci, fun = c("min", "max"), na.rm = TRUE)
```

```
##                                                       min       max
## ts_2017.0802_0808.L4.LE3.CIcyano.MAXIMUM_7day 6.486344e-05 0.0613762
```

Test your function. What is the range of CI values in the transformed raster? Do they make physical sense given the data key?

Questions for Task 1:

Q1.1 What does the CRS of these rasters tell you about how distances and areas are measured? Would you need to reproject before calculating bloom area in square kilometers?

Answer 1.1 The CRS of these rasters is WGS 84 / UTM zone 17N. UTM is a projected coordinate system, which means it flattens the 3D earth onto a 2D grid. Its linear unit of measurement is the meter. This tells you that distances and areas are measured in flat meters.Because the raster is already in a projected coordinate system measured in meters, you do not need to reproject it before calculating area.I would simply calculate the area in square meters and mathematically convert it to square kilometers.

Q1.2 Why is it important to remove sentinel values *before* applying the DN-to-CI transform — rather than after? What would happen to a land pixel (value = 252) if you transformed it without cleaning first?

Answer 1.2 I assume it all comes down to the math. Sentinel values (here like 252 for land) are just arbitrary numeric codes used to label non-data pixels; they aren't real physical measurements. If you apply the mathematical transformation formula before removing them, R will blindly plug the number "252" into the equation. You would accidentally calculate a fake, valid-looking cyanobacteria concentration for dry land, permanently messing your dataset by mixing those fake numbers in with real observations.

Q1.3 The `others = NA` argument in `terra::classify()` is useful here. In your own words, what does it do?

Answer 1.3 When you give terra::classify() your reclassification matrix, R checks every single pixel against the specific rules you set up. If a pixel has a value that falls outside of the intervals you explicitly defined, the others = NA argument tells R to sweep it up and automatically convert it to an NA.

Task 2: Spatial subsetting — crop and mask

2.1 Load and align an area of interest

Load the `bb_erie_sandusky` shapefile from the `bounds/` directory using `sf::read_sf()`. Check its CRS. If it does not match the raster CRS, reproject it using `sf::st_transform()`.

```
#Loading the shapefile using sf
sandusky_aoi <- sf::read_sf("./erie_raster_ex/bounds/bb_erie_sandusky.shp")

#Using the first_raster from Task 1 as our reference for the terra CRS
if (sf::st_crs(sandusky_aoi) != terra::crs(first_raster)) {

  # If they don't match, reprojecting the shapefile to match the raster
  sandusky_aoi <- sf::st_transform(sandusky_aoi, terra::crs(first_raster))
  print("CRS aligned.")

} else {
  print("CRS already match.")
}
```

```
## [1] "CRS aligned."
```

2.2 Crop and mask

Using the **August 16–22, 2017** raster (`ts_2017.0816_0822`):

1. Apply `clean_cicyano()` and `dn_to_ci()` to get a clean, transformed raster
2. Use `terra::crop()` to clip the raster extent to the Sandusky Bay boundary
3. Use `terra::mask()` to set pixels *outside* the boundary to `NA`

Store your final result as `sandusky_0816`.

```
#correct August raster path using string matching
aug_raster_path <- stringr::str_subset(raster_files, "0816_0822")

# Load it
aug_raster <- terra::rast(aug_raster_path)
aug_clean <- clean_cicyano(aug_raster)
aug_ci <- dn_to_ci(aug_clean)

#Crop the raster doum to Sandusky Bay
sandusky_cropped <- terra::crop(aug_ci, sandusky_aoi)

#Mask the raster to exactly the polygon boundary
sandusky_0816 <- terra::mask(sandusky_cropped, sandusky_aoi)
```

2.3 Compare full vs. masked

Using `terra::global()`, calculate and compare the **mean** CI value for:

- The full Lake Erie cleaned/transformed raster
- The Sandusky Bay-masked raster (`sandusky_0816`)

```
#mean for the full Lake Erie raster
print("Full Lake Erie Mean CI:")
```

```
## [1] "Full Lake Erie Mean CI:"
```

```
terra::global(aug_ci, fun = "mean", na.rm = TRUE)
```

```
##                                                      mean
## ts_2017.0816_0822.L4.LE3.CIcyano.MAXIMUM_7day 0.0005582809
```

```
#mean for the masked Sandusky Bay raster
print("Sandusky Bay Mean CI:")
```

```
## [1] "Sandusky Bay Mean CI:"
```

```
terra::global(sandusky_0816, fun = "mean", na.rm = TRUE)
```

```
##                                                    mean
## ts_2017.0816_0822.L4.LE3.CIcyano.MAXIMUM_7day 0.02085237
```

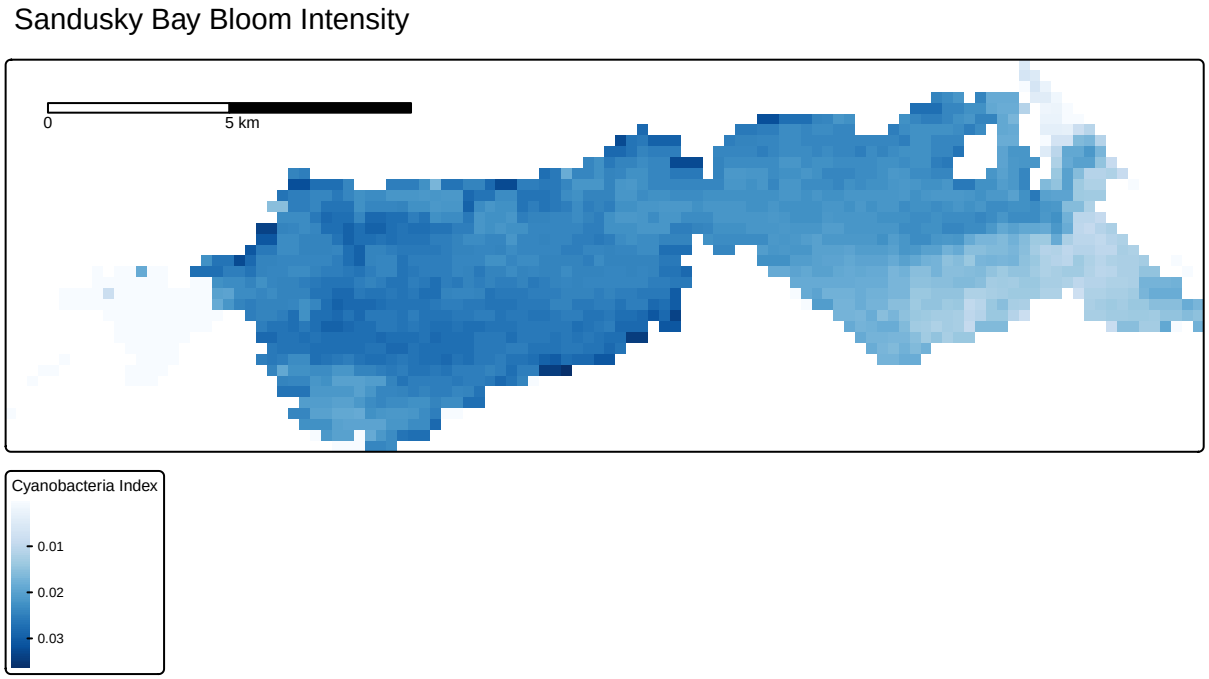
2.4 Map it

Create a `tmap` map of `sandusky_0816`. Your map should include:

- A meaningful title
- A legend with appropriate labels
- A scale bar

```r
tm_shape(sandusky_0816) +
  tm_raster(
    title = "Cyanobacteria Index",
    palette = "Blues",
    style = "cont"
  ) +
  tm_layout(
    main.title = "Sandusky Bay Bloom Intensity",
    main.title.size = 0.9,

    # Legend placement rules
    legend.outside = TRUE,
    legend.outside.position = "right",

    # Legend sizing rules
    legend.title.size = 0.5,
    legend.text.size = 0.4
  ) +
  tm_scale_bar(
    position = c("left", "top"),
    breaks = c(0, 5, 10)
  )
```

```
##
## -- tmap v3 code detected -------------------------------------------------
## [v3->v4] `tm_raster()`: instead of `style = "cont"`, use col.scale =
## `tm_scale_continuous()`.
## i Migrate the argument(s) 'palette' (rename to 'values') to
##   'tm_scale_continuous(<HERE>)'
## [v3->v4] `tm_raster()`: migrate the argument(s) related to the legend of the
## visual variable `col` namely 'title' to 'col.legend = tm_legend(<HERE>)'
## [v3->v4] `tm_layout()`: use `tm_title()` instead of `tm_layout(main.title = )`
## ! `tm_scale_bar()` is deprecated. Please use `tm_scalebar()` instead.
## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "Blues" is named
## "brewer.blues"
```



Questions for Task 2:

Q2.1 In your own words, what is the difference between `terra::crop()` and `terra::mask()`? Why did we use both here instead of just one?

Answer 2.1 terra::crop() shrinks the rectangular bounding box of your raster down to the general area of your shapefile, discarding the rest of the map. However, because rasters must remain rectangular, it still leaves in unwanted pixels outside your specific boundary. terra::mask() acts like a cookie-cutter, keeping the rectangle the same size but turning any pixels that fall outside your exact, irregular polygon shape into NA. We use both for efficiency: cropping first drastically reduces the file size so your computer doesn't have to work as hard during the computationally heavy masking step.

Q2.2 The mean CI value for Sandusky Bay is likely different from the full lake mean. What could explain this? Does this necessarily mean Sandusky Bay has more or less cyanobacteria than average?

Answer 2.2 Sandusky Bay's mean CI is typically much higher than the full lake. Ecologically, this is because the bay is shallow, sheltered, warm, and fed directly by river runoff carrying agricultural nutrients the perfect environment for algae to thrive. So yes, a higher mean indicates that Sandusky Bay inherently experiences significantly more concentrated and severe cyanobacteria blooms than the vast, deeper open waters of Lake Erie.

Task 3: Multi-temporal comparison

3.1 Load and stack multiple rasters

Load and clean **at least four** of the seven rasters (choose dates that span a meaningful time range). Apply `clean_cicyano()` and `dn_to_ci()` to each. Then combine them into a single multi-layer SpatRaster using `c()`.

Tip: using `purrr::map()` over your `raster_files` vector, combined with your preprocessing functions, can make this more efficient.

```r
#Selecting the 4 files
selected_files <- raster_files[c(1, 3, 5, 7)]

#Using purrr::map to load, clean, and transform all 4 files at once
processed_list <- purrr::map(selected_files, ~ {

  # The .x acts as a placeholder for each file path
  r <- terra::rast(.x)
  r_clean <- clean_cicyano(r)
  r_ci <- dn_to_ci(r_clean)

  return(r_ci)
})

#combines the list of rasters into a single multi-layer SpatRaster
raster_stack <- terra::rast(processed_list)

#Renames the layers to make them readable for our final map
names(raster_stack) <- c("Aug_02", "Aug_23", "Sep_27", "Oct_25")

# Print
raster_stack
```

```
## class       : SpatRaster
## size        : 639, 1377, 4  (nrow, ncol, nlyr)
## resolution  : 300, 300  (x, y)
## extent      : 281282.9, 694382.9, 4572609, 4764309  (xmin, xmax, ymin, ymax)
## coord. ref. : WGS 84 / UTM zone 17N (EPSG:32617)
## source(s)   : memory
## varnames    : ts_2017.0802_0808.L4.LE3.CIcyano.MAXIMUM_7day
##               ts_2017.0823_0829.L4.LE3.CIcyano.MAXIMUM_7day
##               ts_2017.0927_1003.L4.LE3.CIcyano.MAXIMUM_7day
##               ...
## names       :       Aug_02,       Aug_23,       Sep_27,       Oct_25
## min values  : 6.486344e-05, 6.486344e-05, 6.486344e-05, 6.486344e-05
## max values  : 6.137620e-02, 6.137620e-02, 5.649370e-02, 4.405549e-02
```

3.2 Temporal summary

Use `terra::global()` on your multi-layer raster to calculate the **mean** CI value per layer (use `na.rm = TRUE`). This will give you one mean value per date.

Organize the results into a data frame with at minimum two columns: `date` and `mean_ci`. You will need to extract the date from each filename. Use `basename()` and/or `stringr::str_extract()` to parse the date string from the file path — the date range is the part of the filename that looks like `0802_0808`.

```r
#Calculating the mean CI for all 4 layers
ci_means <- terra::global(raster_stack, fun = "mean", na.rm = TRUE)

#Extracting the date strings from the filenames
dates_raw <- stringr::str_extract(basename(selected_files), "\\d{4}_\\d{4}")

#Combining them into a clean data frame
temporal_df <- data.frame(
  date = dates_raw,
```

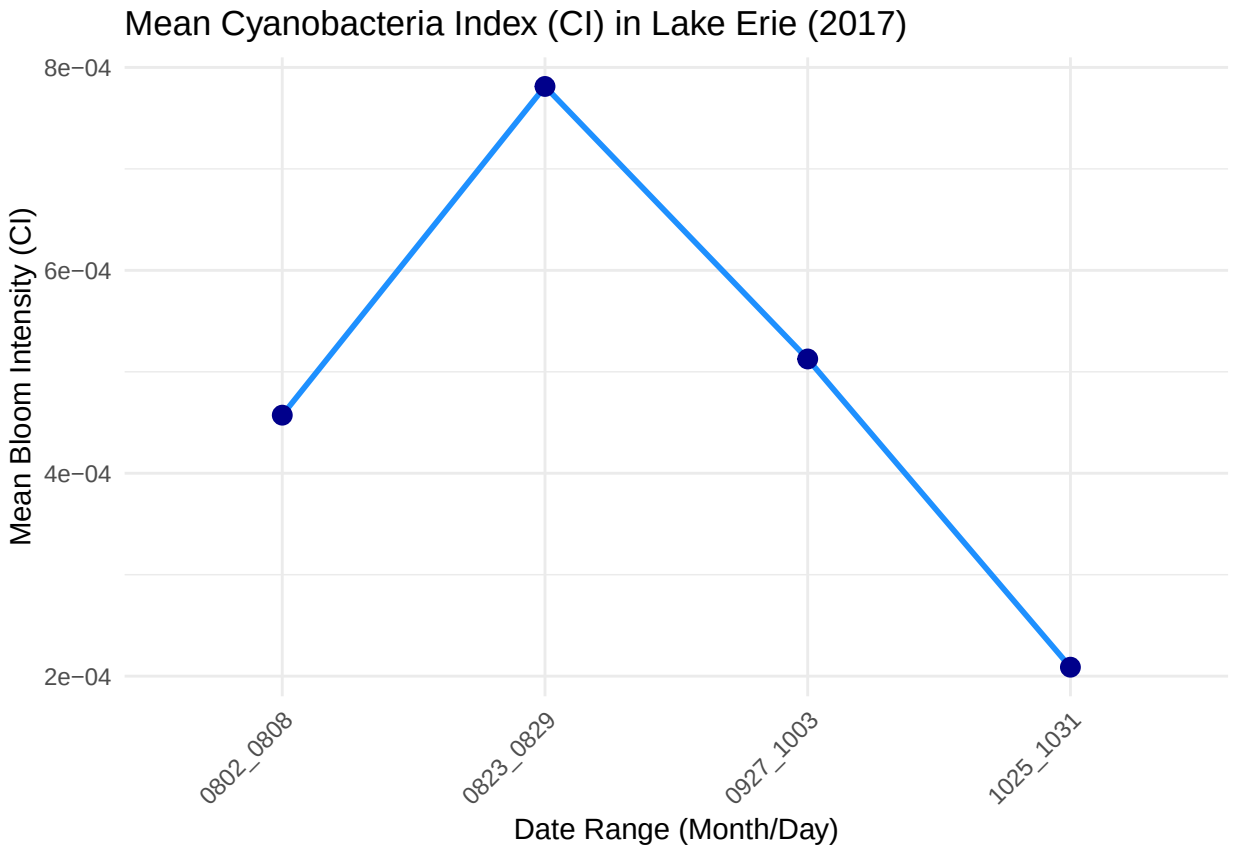
```
  mean_ci = ci_means$mean
)

# Print
print(temporal_df)
```

```
##        date      mean_ci
## 1 0802_0808 0.0004572394
## 2 0823_0829 0.0007812601
## 3 0927_1003 0.0005126261
## 4 1025_1031 0.0002088257
```

3.3 Plot the time series

Using `ggplot2`, create a line or point plot showing mean CI values over time. Label your axes clearly. Which date in your dataset shows the highest mean bloom intensity?

```
library(ggplot2)

#plot
ggplot(temporal_df, aes(x = date, y = mean_ci, group = 1)) +
  geom_line(color = "dodgerblue", linewidth = 1) +
  geom_point(size = 3, color = "darkblue") +
  labs(
    title = "Mean Cyanobacteria Index (CI) in Lake Erie (2017)",
    x = "Date Range (Month/Day)",
    y = "Mean Bloom Intensity (CI)"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```
#The date with the highest mean bloom intensity is 0823_0829 (August 23-29) as per the plot.
```

3.4 Binary bloom comparison

Choose any **two dates** from your dataset. For each, create a binary raster where cells with valid CI values are coded as `1` and all other cells (NA) as `0`. You will need to think about how to do this using `terra::classify()` or logical raster operations.

Use these binary rasters to create two new rasters:

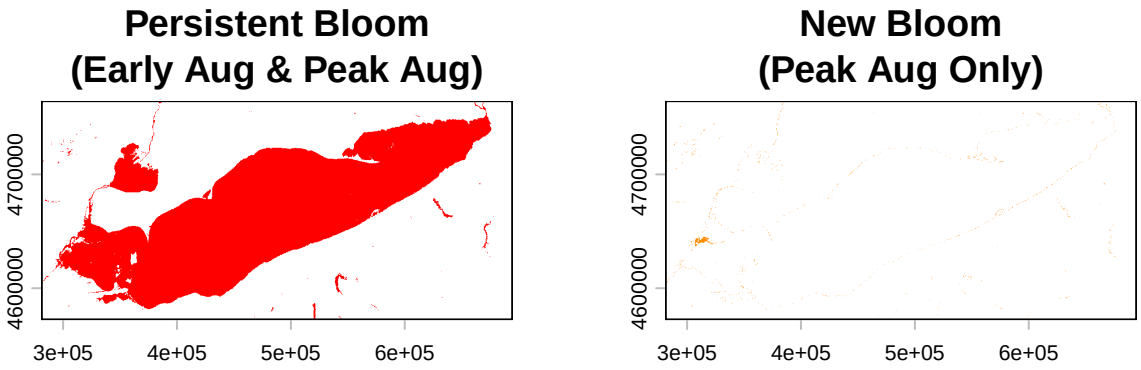
- **Persistent bloom:** cells where blooms were present on *both* dates
- **New bloom:** cells where blooms were present on the *later* date but NOT the earlier date

Map both results.

This requires careful logical thinking before writing code. Plan your approach in pseudocode first.

```
#two dates from multi-layer stack
aug_early <- raster_stack[["Aug_02"]]
aug_peak <- raster_stack[["Aug_23"]]

#Creating Binary Rasters (1 = Bloom, 0 = No Bloom / NA)
early_binary <- !is.na(aug_early)
peak_binary <- !is.na(aug_peak)

# Persistent: Bloom is present in BOTH Early August AND Peak August
persistent_bloom <- (early_binary == 1) & (peak_binary == 1)

# New: Bloom is NOT in Early August, but IS present during the Peak
new_bloom <- (early_binary == 0) & (peak_binary == 1)
```

```r
#Map the results
par(mfrow = c(1, 2)) # Splits the plot window to show 2 maps side-by-side

plot(persistent_bloom,
     main = "Persistent Bloom\n(Early Aug & Peak Aug)",
     col = c("white", "red"),
     legend = FALSE)

plot(new_bloom,
     main = "New Bloom\n(Peak Aug Only)",
     col = c("white", "darkorange"),
     legend = FALSE)
```




```r
par(mfrow = c(1, 1))
```

Questions for Task 3:

Q3.1 What are the limitations of using **mean CI value** as a measure of bloom severity across the whole lake? What would be a more robust alternative metric?

Answer 3.1 Using a whole lake mean can be misleading because it dilutes severe, localized blooms by averaging them with vast areas of clean water.A better approach would be to calculate the Total Bloom Area, the Maximum CI, or the 95th percentile CI, which i believe captures the true intensity without distortion.

Q3.2 Stacking rasters with `c()` in terra creates a multi-layer SpatRaster. What requirement must all rasters meet in order to be stacked this way? (Think: extent, resolution, CRS.)

Answer 3.2 To be stacked using terra::c(), rasters must align perfectly in space. Specifically, every raster in the stack must share the exact same CRS (projection), spatial extent (bounding box), and resolution (grid and pixel size).

Task 4: Zonal analysis with areas of interest

4.1 — Load all boundary files

Use `list.files()` to get all `.shp` files from the `bounds/` directory. Load each with `sf::read_sf()`, then combine them into a single sf object using `dplyr::bind_rows()`. Make sure each polygon has an identifier — if not, add a column with the filename or AOI name.

```r
library(sf)
library(dplyr)
library(tools)
```

```
##
## Attaching package: 'tools'
```

```
## The following object is masked from 'package:tmap':
##
##     toTitleCase
```

```
#Get the list of files
bounds_files <- list.files("bounds/", pattern = "\\.shp$", full.names = TRUE)

target_crs <- terra::crs(raster_stack)
sf_list <- lapply(bounds_files, function(file_path) {

  aoi <- read_sf(file_path)

  aoi <- st_zm(aoi, drop = TRUE)

  aoi <- st_transform(aoi, target_crs)

  clean_aoi <- st_sf(
    aoi_name = file_path_sans_ext(basename(file_path)),
    geometry = st_geometry(aoi)
  )

  return(clean_aoi)
})

#Bind clean list together
all_aois <- do.call(rbind, sf_list)

# Print
print(all_aois)
```

```
## Simple feature collection with 6 features and 1 field
## Geometry type: POLYGON
## Dimension:     XY
## Bounding box:  xmin: 283549.5 ymin: 4586777 xmax: 375219.9 ymax: 4663444
## Projected CRS: WGS 84 / UTM zone 17N
##                aoi_name                       geometry
## 1       bb_erie_islands POLYGON ((364557.6 4586783,...
## 2         bb_erie_large POLYGON ((374867.5 4640842,...
## 3        bb_erie_maumee POLYGON ((297096.1 4631671,...
## 4 bb_erie_sandusky_plus POLYGON ((364479.5 4586809,...
## 5      bb_erie_sandusky POLYGON ((359172.9 4594873,...
## 6   bb_erie_west_sister POLYGON ((324315.5 4609049,...
```

4.2 Extract by area

Using the date you identified in Task 3.3 as having the highest mean bloom intensity, extract CI values for each area of interest using `terra::extract()`. Calculate the **mean** and **maximum** CI per AOI.

Tip: `terra::extract()` returns a data frame. The `ID` column links rows to the AOI that was used for extraction. You can use `group_by()` and `summarize()` to aggregate.

```
library(dplyr)
```

```r
#peak bloom raster identified in Task 3.3 (August 23)
peak_raster <- raster_stack[["Aug_23"]]

#Extract CI values for each area of interest (AOI)
extracted_ci <- terra::extract(peak_raster, all_aois)

#Calculate the mean and maximum CI per AOI
zonal_summary <- extracted_ci %>%
  group_by(ID) %>%
  summarize(
    mean_ci = mean(Aug_23, na.rm = TRUE),
    max_ci = max(Aug_23, na.rm = TRUE)
  ) %>%

  mutate(aoi_name = all_aois$aoi_name[ID]) %>%

  select(aoi_name, mean_ci, max_ci)

# Print
print(zonal_summary)
```

```
## # A tibble: 6 x 3
##   aoi_name                mean_ci max_ci
##   <chr>                     <dbl>  <dbl>
## 1 bb_erie_islands       0.000371 0.0240
## 2 bb_erie_large         0.00502  0.0614
## 3 bb_erie_maumee        0.0335   0.0614
## 4 bb_erie_sandusky_plus 0.0122   0.0441
## 5 bb_erie_sandusky      0.0241   0.0441
## 6 bb_erie_west_sister   0.00114  0.0172
```

4.3 Summary table

Present your results as a clean summary table. Which AOI had the highest mean CI value on that date? Which had the highest maximum?

```r
# Creating a clean summary table using knitr
knitr::kable(zonal_summary,
             col.names = c("AOI Name", "Mean CI", "Max CI"),
             digits = 6,
             caption = "Peak Bloom Severity by Area of Interest (Aug 23, 2017)")
```

Table 1: Peak Bloom Severity by Area of Interest (Aug 23, 2017)

AOI Name	Mean CI	Max CI
bb_erie_islands	0.000371	0.023988
bb_erie_large	0.005025	0.061376
bb_erie_maumee	0.033485	0.061376
bb_erie_sandusky_plus	0.012239	0.044055
bb_erie_sandusky	0.024068	0.044055
bb_erie_west_sister	0.001140	0.017219

```r
highest_mean <- zonal_summary$aoi_name[which.max(zonal_summary$mean_ci)]
highest_max <- zonal_summary$aoi_name[which.max(zonal_summary$max_ci)]

cat("The AOI with the highest mean CI is:", highest_mean, "\n")
```

```
## The AOI with the highest mean CI is: bb_erie_maumee
```

```r
cat("The AOI with the highest max CI is:", highest_max, "\n")
```

```
## The AOI with the highest max CI is: bb_erie_large
```

4.4 Map with AOI overlays

Create a `tmap` map showing the CI raster for your chosen date with the AOI boundaries overlaid. Use `tm_borders()` for the boundaries and label them if possible.

```r
library(tmap)
tmap_mode("plot")
```

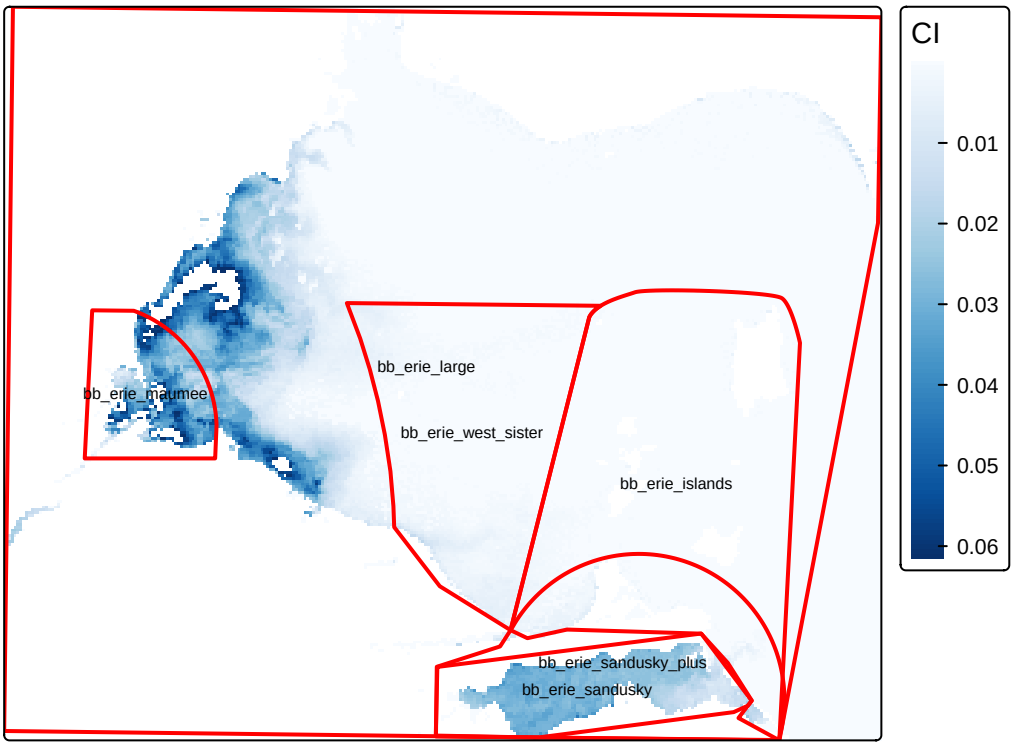
```
## i tmap modes "plot" - "view"
## i toggle with `tmap::ttm()`
```

```r
aoi_centroids <- sf::st_centroid(all_aois)
```

```
## Warning: st_centroid assumes attributes are constant over geometries
```

```r
tm_shape(peak_raster, bbox = sf::st_bbox(all_aois)) +
  tm_raster(title = "CI", palette = "Blues", style = "cont") +

tm_shape(all_aois) +
  tm_borders(col = "red", lwd = 2) +

tm_shape(aoi_centroids) +
  tm_text("aoi_name",
          size = 0.5,
          bg.color = "white",
          bg.alpha = 0.7,
          auto.placement = TRUE) +

tm_layout(main.title = "Peak Bloom & AOIs",
          legend.outside = TRUE)
```

```
##
## -- tmap v3 code detected -------------------------------------------------
## [v3->v4] `tm_raster()`: instead of `style = "cont"`, use col.scale =
## `tm_scale_continuous()`.
## i Migrate the argument(s) 'palette' (rename to 'values') to
##   'tm_scale_continuous(<HERE>)'[v3->v4] `tm_raster()`: migrate the argument(s) related to the legend
## visual variable `col` namely 'title' to 'col.legend = tm_legend(<HERE>)'[v3->v4] `tm_text()`: migrat
## 'point.label') to 'options = opt_tm_text(<HERE>)'[v3->v4] `tm_layout()`: use `tm_title()` instead of
## `cols4all::c4a_gui()` to explore them. The old palette name "Blues" is named
## "brewer.blues"
```



Questions for Task 4:

Q4.1 You may have gotten `NA` values in your `terra::extract()` results for some AOIs. What could cause this? Name at least two reasons.

Answer 4.1. NA values during spatial extraction usually happen for two main reasons. First, the raster may already be masked during preprocessing pixels affected by clouds, land, or sensor errors are set to NA so they don't bias the results. Second, there can be a spatial mismatch, where parts of an AOI extend beyond the area covered by the satellite image, meaning there are simply no raster values available in those regions.

Q4.2 Looking at the spatial distribution of your AOIs on the map, which areas do you expect to be most prone to severe blooms, and why? (Consider geography, hydrology, land use in the Lake Erie watershed.)

Answer 4.2. The areas most likely to experience severe blooms are the western basin of Lake Erie, especially around Maumee Bay, and also Sandusky Bay. These areas are shallow and more enclosed, so the water warms up quickly and tends to stagnate in summer, which favors algal growth. In addition, they receive large amounts of nutrient runoff from surrounding agricultural land, particularly phosphorus and nitrogen. This combination of warm, slow moving, nutrient rich water makes them especially prone to intense cyanobacteria blooms.

Questions

1. Throughout this lab you have been working with **raster** data. In what ways was the analysis workflow different from working with vector data (like you did in Labs 1–3)? What concepts translated directly, and what was fundamentally different?

Answer 1: Both raster and vector analyses require matching coordinate reference systems and making sure spatial layers line up correctly, so that part translates directly. The main difference comes from how the data are structured and analyzed. Vector data deals with discrete features (points, lines, polygons) and is

typically handled through attribute tables and operations like joins or intersections. Raster data, on the other hand, represents continuous surfaces as grids of pixels. Because of that, raster analysis relies more on cell-based calculations, such as map algebra, reclassification, and working with pixel values directly, rather than table-based queries.

2. Cyanobacteria blooms are affected by temperature, nutrient runoff, and lake circulation. Based on the temporal patterns you observed in Task 3, does the timing of peak bloom match what you would expect ecologically? Explain your reasoning (I know you're not an expert on blooms - just demonstrate thinking).

Answer 2: The late August peak is consistent with what you would expect ecologically. By that time of year, water temperatures are at their highest, which promotes cyanobacteria growth. At the same time, nutrients from spring and early summer runoff, especially nitrogen and phosphorus from agriculture have had time to accumulate in the system. Combined with slower water movement in late summer, particularly in the western basin, these conditions would create an environment that strongly favors large blooms.

3. We used `terra::extract()` in Task 4 to perform a zonal summary. How does this relate conceptually to the **zonal operations** category of map algebra discussed in class?

Answwer 3: Using terra::extract() is an example of a zonal operation from map algebra. In this case, the AOI polygons define the zones, and the function summarizes the raster values (like calculating a mean) within each zone. So instead of looking at individual pixels, we are grouping them by area and then calculating statistics for each group, which is exactly what zonal operations are designed to do